On this page

Key-pair authentication

Overview

Our quick start guide shows how to set up the recordSDK using a public app ID for SDK Standard apps.

SDK Custom apps, however, require key-pair authentication—a more secure form of initialization. SDK Standard apps can also be configured to use this type of authentication.

The process uses signed JSON Web Tokens (JWS) to verify a client has been authorized by the application owner using cryptographic keys. A private key is stored on your server and Loom uses the corresponding public key to verify incoming tokens.

Setting up for SDK Custom

Developer portal onboarding

If you create an SDK Custom app through the initial onboarding form, a key-pair will automatically be created and a PEM file containing your private key should appear in your downloads with a file name matching your application.

Developer dashboard "Create application"

If creating an SDK Custom app through the developer dashboard, submitting the form will trigger a key-pair to be created and a PEM file containing your private key should appear in your downloads with a file name matching your application.

Setting up for SDK Standard

To provision a key-pair for an SDK Standard app, follow these steps:

1. Go to the Developer Portal.
2. Click `Update` on the application for which you would like to enable key-pair authentication.
3. Press the `Create private key` button.

A PEM file containing your private key should appear in your downloads with a file name matching your application.

> **WARNING**
>
> The private key file that gets downloaded must be kept secret and is not recoverable. Be sure to save it somewhere safe!

Using your private key

With the private key saved, you can now sign the tokens required to load the recordSDK.

JSON Web Token

The token you'll send contains information we'll use to verify your signature and validate requests to initialize the recordSDK. Each token is intended to last just long enough for your client to set up the recordSDK.

Required Token Payload

```
{ // Header
  "alg": "RS256"
}
{ // Payload
  "iat": 1639493265,
  "iss": "<PUBLIC_APP_ID>",
  "exp": 1639493385
}
```

Header

Property	Description
`alg`	This specifies the algorithm to be used while verifying the signature. We only support `RS256`.

Payload

Property	Description
`iat`	The unix time at which the token was provisioned. This is used to enforce a max lifespan of incoming tokens.
`iss`	The token issuer. This should be set to the public app ID of your application.
`exp`	The unix time at which the token expires. It's recommended to maintain a short timespan (3 minutes or less).

Server-side implementation

The following example of creating a JWS uses `jose` but any JSON web token + signature library for your runtime will work.

```
import * as jose from 'jose';

declare const PRIVATE_PEM: string;

const privateKey = await jose.importPKCS8(PRIVATE_PEM, "RS256");
const jws = await new jose.SignJWT({})
  .setProtectedHeader({ alg: "RS256" })
  .setIssuedAt()
  .setIssuer("2a8e4925-3996-44f5-85e0-1dc19d5f4c85")
  .setExpirationTime("2m")
  .sign(privateKey);
```

From here `jws` can be attached to the page response and used to initialize the SDK.

Client-side implementation

For the client, the only change will be to supply the signed JWT to the new `jws` option instead of using public app ID directly.

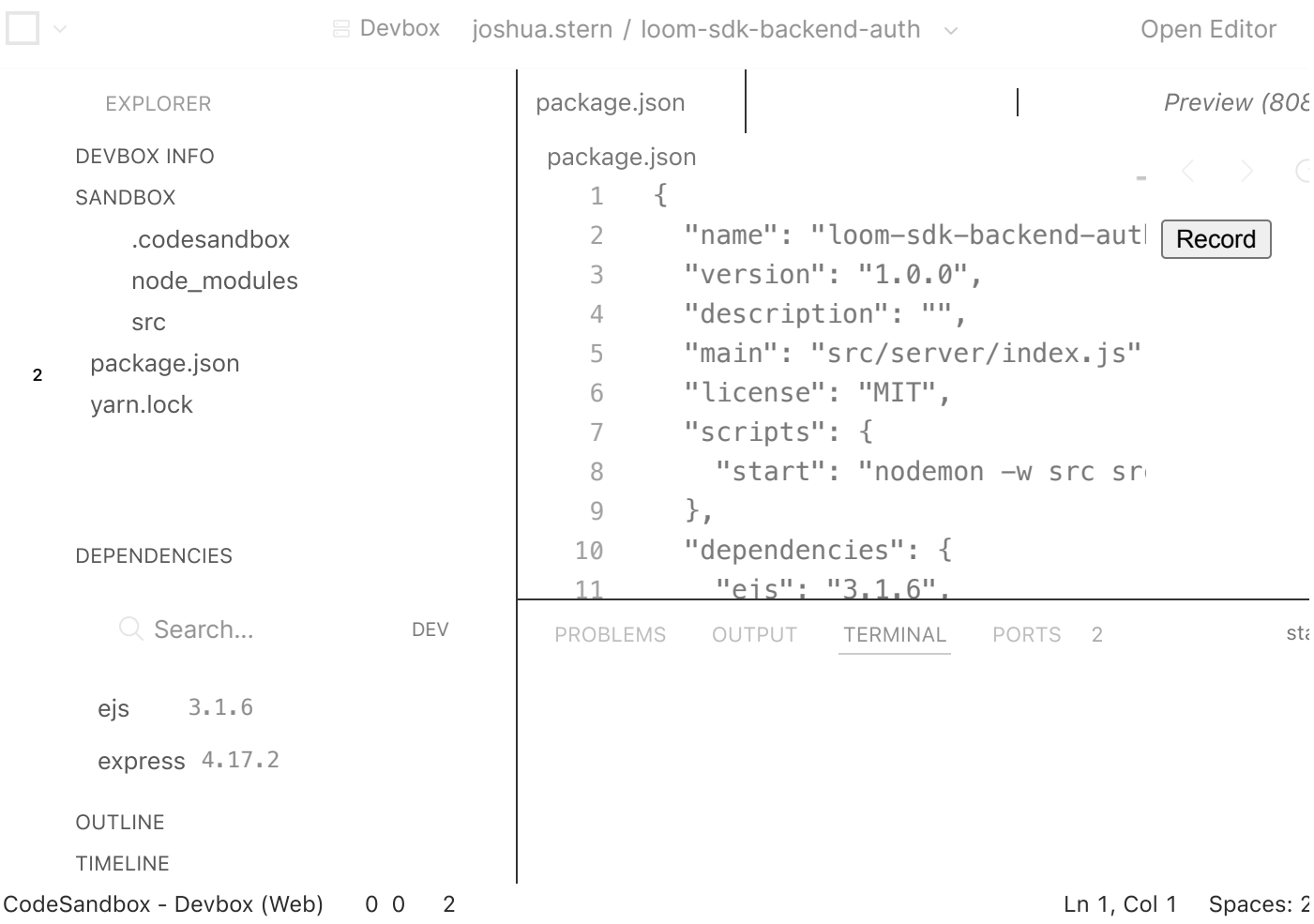
```ts
import { setup } from '@loomhq/record-sdk';

// Assuming jws has been fetched from your server
declare const serverJws: string;

await setup({
  jws: serverJws,
});
```

> NOTE
>
> If you dynamically load `@loomhq/record-sdk` then you'll need to make sure your client can also request a valid token when it's needed.

Example


Devbox
joshua.stern / loom-sdk-backend-auth
Open Editor
EXPLORER
DEVBOX INFO
SANDBOX
.codesandbox
node_modules
src
package.json
yarn.lock
DEPENDENCIES
Search...
DEV
ejs 3.1.6
express 4.17.2
OUTLINE
TIMELINE
package.json
Preview (808
package.json
{
"name": "loom-sdk-backend-aut
"version": "1.0.0",
"description": "",
"main": "src/server/index.js"
"license": "MIT",
"scripts": {
"start": "nodemon -w src sr
},
"dependencies": {
"ejs": "3.1.6",
Record
PROBLEMS
OUTPUT
TERMINAL
PORTS 2
CodeSandbox - Devbox (Web)
0 0 2
Ln 1, Col 1
Spaces: 2